

Système d'élaboration de l'architecture du système de gestion d'un essaim de drones

Description détaillée et complète d'un système existant et opérationnel

OBJET	Chaîne veille scientifique → analyse d'impact → modèle LikeC4 → publication → gouvernance
NATURE	Reverse engineering sur pièces (fichiers, scripts, états, service en production)
VERSION	1.0
DATE	22 septembre 2026
STATUT	Vérifié — site likec4.breizh.ai en production (HTTP 200)
CONFIDENTIALITÉ	Interne

Document établi par analyse directe du système — aucune donnée inventée.
Les identifiants sensibles sont remplacés par [REDACTED].

Spécification détaillée — Système d'élaboration de l'architecture d'un système de gestion d'un essaim de drones

Version : 1.0 **Date de rédaction** : 22 septembre 2026 **Objet** : description exhaustive du système existant, tel qu'observé sur le poste, sans extrapolation **Méthode** : exploration directe du système de fichiers, des dépôts Git, des bases d'état et des services en exécution

Avertissement de méthode

Ce document décrit un système **réel et en fonctionnement**. Chaque affirmation structurelle (chemin, nom de fichier, commande, compteur) a été vérifiée par lecture directe du système au 22 septembre 2026. Les éléments qui relèvent de l'interprétation sont explicitement marqués comme tels.

Aucune clé d'API, aucun jeton, aucun mot de passe, aucune chaîne de connexion n'apparaît dans ce document. Lorsqu'un fichier contenant des identifiants est mentionné, seule sa présence est signalée et sa valeur est remplacée par [REDACTED].

1. Vue d'ensemble

1.1 En une phrase

Le système transforme de la **littérature scientifique** en une **architecture logicielle formalisée, vérifiable et publiée** pour un essaim de 30 plateformes autonomes (26 aériennes + 4 de surface).

Autrement dit : il collecte des articles, en extrait ce qui est *prouvé*, le confronte à un modèle d'architecture, signale les écarts, et publie le tout sur le Web — en refusant explicitement de présenter comme acquis ce qui ne l'est pas.

1.2 Le problème que le système résout

Concevoir un essaim de drones impose de répondre à des questions de recherche (consensus sous brouillage, allocation de tâches, fusion de capteurs) dont les réponses sont dispersées dans la littérature, souvent partielles, parfois contradictoires. Le risque n'est pas de manquer d'articles : c'est de **croire qu'une question est résolue alors qu'elle ne l'est pas**.

Le système attaque ce risque par trois moyens :

1. **Automatiser la veille** — un pipeline hebdomadaire interroge arXiv, Crossref, OpenAlex et HuggingFace, filtre, déduplique et archive.
1. **Relier chaque affirmation à une source** — ou la déclarer non prouvée.

C'est l'invariant INV-5 du modèle.

1. **Rendre les écarts visibles** — les questions sans réponse sont inscrites dans le modèle comme `GAP-*`, pas passées sous silence.

1.3 Fonctions couvertes

Le système couvre sept chaînes fonctionnelles, chacune détaillée dans une section dédiée :

#	Chaîne	Ce qu'elle fait	Section
1	Traitements planifiés	Déclenche la veille, la gouvernance et le contrôle de dérive	§3
2	Veille scientifique	Recherche, sélection, déduplique et stocke les articles	§4
3	Gestion de versions	Versionne le profil, les algorithmes et le modèle	§5
4	Fichiers LikeC4	Élabore et stocke les sources du modèle	§6
5	Schémas LikeC4	Compose les vues et diagrammes du modèle	§7
6	Publication	Sert le modèle sur likec4.breizh.ai	§8
7	Spécifications et algorithmes	Gère les dépôts GitHub et leurs submodules	§9

1.4 Objectifs

Objectif principal — maintenir une architecture d'essaim dont chaque affirmation est soit **adossée à une source scientifique**, soit **explicitement déclarée non prouvée**.

Objectifs dérivés, tels qu'inscrits dans le système :

- **Sélectionner les articles pertinents** — ne pas accumuler, mais retenir ce qui éclaire une décision d'architecture.
- **Produire un modèle LikeC4 exécutable** — le modèle est compilé et validé, pas seulement rédigé.
- **Publier une architecture consultable** — accessible en ligne, à jour.
- **Détecter les régressions silencieuses** — perte d'éléments, de relations ou de vues doit échouer bruyamment, pas passer inaperçue.
- **Rendre la chaîne falsifiable** — chaque promesse du modèle porte un test qui échoue quand elle est violée.

1.5 Valeur ajoutée

Ce qui distingue ce système d'un simple dépôt de documentation :

- **La traçabilité est structurelle, pas déclarative.** Le modèle ne *décrit* pas sa chaîne de consommation : il l'exécute et la teste. Un modèle qui se contente d'exporter du JSON est une promesse ; ici, la promesse porte un test.
- **L'incertitude est un objet de premier ordre.** Les 7 `GAP-*` du modèle

nomment les questions ouvertes. Plusieurs portent la mention « AUCUN ARTICLE TROUVÉ » — c'est une information d'architecture, pas un échec à masquer.

- **La gouvernance est outillée.** Trois contrôles automatiques (validation, non-régression, divergence) protègent le modèle contre les pertes silencieuses.
- **La veille est continue et sans intervention.** Le pipeline tourne seul ; le corpus s'enrichit sans action manuelle.
- **La doctrine est explicite.** Le principe fondateur — *LikeC4 est la source de vérité sémantique ; Blender et le moteur 3D en sont des consommateurs* — est inscrit dans le dépôt, pas dans la tête de son auteur.

1.6 Angles morts

Un système qui ne nomme pas ses limites les subit. Ceux-ci sont **assumés et documentés** dans le dépôt lui-même.

Limites de périmètre — ce que le système n'est pas :

- **Ce n'est pas un simulateur opérationnel.** Il démontre une méthode : comment un modèle sémantique devient une chaîne de consommation vérifiable.
- **Aucune dynamique de vol.** Les formations sont illustratives (anneaux déterministes montrant la population) ; aucun modèle aérodynamique.
- **Aucune temporalité.** Pas d'horloge de simulation, pas d'état qui évolue.
- **Aucune coordonnée 3D dans le modèle.** C'est l'invariant INV-4 : la spatialisation appartient au consommateur, pas au modèle sémantique.
- **Les assets 3D ne sont pas chargés par le viewer.** La chaîne procédurale et la chaîne Blender (7 GLB traçables) existent séparément ; leur intégration n'est pas faite.
- **Le rendu validé est logiciel** (SwiftShader) — un GPU réel n'a pas été testé.
- **La 2D de repli n'a été exercée qu'en test**, jamais sur une machine réellement dépourvue de WebGL.

Questions scientifiques ouvertes — les 7 **GAP-* du modèle :**

GAP	Question ouverte	État
GAP-1	CBBA à 30 sous partition persistante + brouillage	Locale prouvée, globale = hypothèse
GAP-2	CBF hétérogène au-delà de 30 en matériel réel	Aucun article trouvé
GAP-3	CRDT (LWW + OR-Set) sur drones en vol, brouillage RF réel	Convergence d'état prouvée ; mission non testée
GAP-4	Fusion EKF/UKF hiérarchique à ≥ 100 plateformes	Aucun article trouvé
GAP-5	Brouillage réel sur un essaim de 30 en vol	Non démontré
GAP-6	Localisation coopérative à 30 plateformes en vol réel	Non démontré
GAP-7	Biais appris et DMG	Ouvert

Limite structurelle majeure — l'invariant **INV-3** : *la validation à 30 plateformes reste non prouvée*. Le système ne prétend pas le contraire.

Angle mort de méthode — le corpus scientifique est collecté automatiquement, mais sa **pertinence n'est pas validée par un humain à chaque cycle**. Le système filtre et déduplique ; il ne juge pas de la qualité scientifique d'un article. Un article médiocre mais bien indexé entre dans le corpus.

1.7 Principe directeur

Inscrit dans la configuration du modèle :

« *Le contrat de mission est embarqué ; le lien est un bonus, pas une dépendance.* »

Et, au niveau du dépôt :

« *LikeC4 est la source de vérité sémantique. Blender et le moteur 3D en sont des consommateurs.* »

1.8 Cartographie des répertoires de travail

Tous les répertoires de travail du système sont décrits en §2.

2. Répertoires de travail

2.1 Vue synthétique

Répertoire	Rôle	Versionné	Taille indicative
~/hermes/profiles/swarm-drone/	Profil Hermes installé (exécution)	Non (copie)	~4,0 Go
~/swarmdrone-profile/	Source de vérité du profil	Oui (GitHub privé)	~1,2 Mo
~/swarmdrone-research/	Corpus scientifique et état de veille	Non	~815 Mo (hors PDF) + 5,4 Go de PDF
~/workspace/swarm-drones_likec4/	Modèle LikeC4 et outillage	Oui (local)	~37 Mo (hors .git)
~/workspace/alg-protocole/	Spécification du protocole H-Zip	Oui (GitHub public)	~536 Ko
~/hermes/cron/	Planificateur et état des tâches	Non	—
~/hermes/scripts/	Scripts des tâches planifiées	Partiellement	—
/docker/likec4/workspace/	Répertoire servi par le conteneur	Non (bind mount)	—

2.2 `~/hermes/profiles/swarmdrone/` — profil installé

C'est le **répertoire d'exécution**. Il contient le profil Hermes tel qu'installé, avec ses scripts, ses compétences, sa configuration et son état.

Éléments notables :

- `scripts/` — 31 scripts exécutables (`.py` / `.sh`) plus 2 entrées annexes (`__pycache__` / et `read-global-deepinfra-secret` , sans extension)
- `skills/` — 31 compétences : 18 `swarm-*` issues de la source versionnée

plus 13 compétences génériques fournies par Hermes (builtin), présentes uniquement dans la copie installée

- `references/` — politique scientifique et taxonomies
- `cron/jobs.json` — définition de la tâche planifiée du profil
- `config.yaml` — configuration du profil (contient des identifiants `[REDACTED]`)
- `distribution.yaml` — manifeste de distribution (chemins possédés)
- `state.db` — base d'état Hermes (~38 Mo)

Point critique : ce répertoire est une **copie**. Il est écrasé par `hermes profile update swarmdrone` pour tous les chemins déclarés dans `distribution.yaml`. Toute modification faite directement ici est perdue à la prochaine mise à jour.

2.3 `~/swarmdrone-profile/` — source de vérité

Dépôt Git contenant la **définition versionnée** du profil. C'est ici que les modifications doivent être faites.

- `distribution.yaml` — déclare les 30 chemins possédés par la distribution
- `setup.sh` — installeur idempotent (`hermes profile install "$ROOT_DIR" --force`)
- `scripts/` — 31 scripts versionnés (dont `check_profile_drift.py`, `watchdog_derive_profil.sh`)
- `skills/` — 18 compétences `swarm-*` (les 13 compétences génériques de Hermes

ne sont pas versionnées ici : elles sont fournies par le runtime)

- `references/` — documents de référence
- `tests/` — 19 tests déterministes (dont `test_pipeline_script_references.py` ,

garde-fou contre la perte silencieuse de scripts)

- `AGENTS.md` — carte du dépôt et règle de synchronisation

Dépôt distant : <https://github.com/dagornc/swarmdrone-profile> (privé), branche `master` , 10 commits au 22/09/2026.

2.4 `~/swarmdrone-research/` — corpus scientifique

Répertoire **externe au profil**, contenant le corpus et l'état de la veille.

Élément	Contenu
articles/	2 139 enregistrements Markdown numérotés
pdf/	797 fichiers PDF
excluded/	111 enregistrements écartés
history/	70 enregistrements historiques
catalog.json	Catalogue complet (~2,0 Mo)
INDEX.md	Index lisible (~258 Ko)
state.json	État de numérotation et de veille
quality-ledger.json	Registre qualité (200 entrées)
quality-summary.json	Synthèse qualité
impact-proposals.json	Propositions d'impact (~183 Ko)
impact-state.json	État de l'analyse d'impact
corpus-link-audit.json	Audit corpus ↔ liens LikeC4
logs/	Journaux d'exécution horodatés
.writer.lock, .run.lock, .quality.lock	Verrous de concurrence

État vérifié au 22/09/2026 : `next_number` = 2251 ; dernière publication revendiquée = 2026-09-20 ; dernière recherche = 2026-09-03 → 2026-09-21. *Le run hebdomadaire du 22/09 (11:38 → 11:46 UTC, 455 s, 7 étapes, succès) a porté le corpus à 2 139 articles.*

2.5 ~/workspace/swarmdrones_likec4/ — modèle LikeC4

Dépôt Git **local** (sans dépôt distant) contenant le modèle et son outillage.

- 18 fichiers `.c4` — le modèle (source de vérité)
- `tools/` — pipeline d'export, contrôles qualité, générateur d'assets
- `viewer/` — consommateur 3D et son test de logique
- `docs/` — 18 documents de spécification
- `export/` — sorties d'export (non versionnées)
- `public/` — fichiers servis publiquement
- `backup_v1/ ... backup_v21_final/` — 22 sauvegardes manuelles (non versionnées)
- `sync_likec4.sh` — script de synchronisation vers le répertoire servi
- `gouvernance_swarmdrones.py` — garde-fou de gouvernance
- `gov_baseline.json` — baseline de non-régression
- `.git/hooks/post-commit` — synchronisation automatique après commit

2.6 ~/.hermes/cron/ — planificateur

- `jobs.json` — définition des tâches du profil `default`
- `executions.db` — historique durable des exécutions
- `deliveries.db` — historique des notifications
- `output/` — sorties des tâches

- `ticker_heartbeat`, `ticker_last_success` — battements du planificateur

2.7 `/docker/likec4/workspace/` — répertoire servi

Répertoire monté dans le conteneur `likec4` en tant que `/data` (bind mount). C'est ce que le site public sert réellement. Il est alimenté par `sync_likec4.sh` depuis `~/workspace/swarm-drones_likec4/`.

3. Traitements planifiés (cron)

3.1 Architecture du planificateur

Hermes embarque un planificateur de tâches persistant. Chaque profil possède son propre fichier `cron/jobs.json`. Les tâches sont exécutées par un « ticker » dont la vivacité est attestée par `~/hermes/cron/ticker_heartbeat`.

Deux modes d'exécution existent :

- **mode agent** — le script produit une sortie qui est injectée dans le prompt

d'un agent, lequel raisonne puis agit ;

- **mode no-agent** — le script *est* la tâche ; sa sortie standard est

délivrée telle quelle. Une sortie vide signifie « silence » (aucune notification). C'est le **motif watchdog**.

3.2 Les trois tâches du système

Nom	Profil	Planification	Script	Mode	Livraison	État
Derive profil Swarm-Drone	default	0 7 *	<code>watchdog_derive_profil.sh</code>	no-agent	origin	ok
Gouvernance Swarm-Drones	default	0 8 *	<code>watchdog_gouvernance.sh</code>	no-agent	telegram	ok
swarmdrone-weekly-corpus	swarm-drone	0 8 1	<code>swarmdrone-corpus-update.sh</code>	no-agent	local	ok

Identifiants : `d18bf1f3b12a` (dérive), `709bfab177b1` (gouvernance), `8c56b0f01dbf` (corpus hebdomadaire).

3.3 Tâche « Derive profil SwarmDrone »

Objet : détecter toute divergence entre la source versionnée (`~/swarmdrone-profile/`) et la copie installée (`~/hermes/profiles/swarmdrone/`).

Fonctionnement : enveloppe mince autour de `check_profile_drift.py`, qui compare les 30 chemins `distribution_owned` par empreinte SHA-256.

Comportement : silencieux si aucune dérive (code 0, sortie vide) ; alerte nommant le chemin divergent si dérive (code 1).

Raison d'être : le 21 septembre 2026, un correctif de concurrence des collecteurs avait été appliqué uniquement à la copie installée. La mise à jour suivante l'aurait effacé silencieusement. Cette tâche détecte la dérive dès qu'elle apparaît.

3.4 Tâche « Gouvernance SwarmDrones »

Objet : protéger le modèle LikeC4 contre les régressions silencieuses.

Fonctionnement : enveloppe autour de `gouvernance_swarmdrones.py`, qui exécute trois contrôles :

1. **Validation** — `likec4 validate` (compilation réelle). Bloquant.
2. **Non-régression** — comparaison des compteurs (éléments, relations, vues, par type) avec `gov_baseline.json`. Toute baisse est un échec. Bloquant.

1. **Divergence** — comparaison des fichiers `.c4` entre la copie active (`/docker/likec4/workspace`) et la copie Git. Avertissement non bloquant.

Comportement : silencieux si les trois contrôles passent ; alerte sinon.

3.5 Tâche « swarmdrone-weekly-corpus »

Objet : mettre à jour le corpus scientifique chaque lundi à 08:00.

Fonctionnement : `swarmdrone-corpus-update.sh` appelle `weekly_corpus_update.py`, qui orchestre un pipeline de sept étapes (§4.5).

Concurrence : le pipeline est protégé par un **verrou de run** (`.run.lock`). Si un run est déjà en cours, le nouveau run s'arrête immédiatement avec `skipped: true` et `skip_reason: run_lock_busy` — sans erreur. Le verrou d'écriture du corpus (`.writer.lock`) n'est tenu que pendant la phase d'écriture de chaque collecteur ; la découverte réseau s'effectue hors verrou.

Délais : `--lock-timeout` = 300 s ; `--step-timeout` = 900 s.

3.6 Historique et traçabilité

Les exécutions sont enregistrées dans `~/hermes/cron/executions.db` (table `executions`) et consultables par `hermes cron runs <id>`. Les incidents durables sont listés par `hermes cron incidents`.

4. Recherche, sélection et stockage des articles scientifiques

4.1 Objet

Constituer et maintenir un corpus de littérature scientifique primaire pertinente pour la conception d'essaims de drones, avec numérotation stable, traçabilité de provenance et contrôle qualité bibliographique.

4.2 Les trois collecteurs

Collecteur	Source	Rôle
<code>swarm-corpus.py</code>	arXiv	Collecte primaire, numérotation, catalogue
<code>swarm-scholarly-corpus.py</code>	Crossref, OpenAlex	Enrichissement métadonnées
<code>swarm-hf-corpus.py</code>	Hugging Face	Découverte de modèles et jeux de données

4.3 `swarm-corpus.py` — collecteur primaire

Sous-commandes :

- `update` — découvre et stocke les enregistrements arXiv récents
- `ingest-arxiv` — archive des articles découverts, hors fenêtres hebdomadaires
- `import` — importe des enregistrements Markdown numérotés
- `rebuild` — reconstruit le catalogue et l'index depuis `articles/`
- `audit` — audite la numérotation et la provenance
- `reconsider` — réévalue les exclusions anciennes
- `review-hf` — examine le périmètre Hugging Face

Fonctions internes notables : `arxiv_entries()` (découverte réseau), `extract_pdf()` (récupération du PDF), `add_arxiv()` (écriture numérotée), `taxonomy()` (classification), `trust_for()` (évaluation de la confiance).

Numérotation : chaque enregistrement reçoit un numéro stable et croissant, suivi dans `state.json` (`next_number`). Au 22/09/2026 : 2208.

4.4 `swarm-scholarly-corpus.py` et `swarm-hf-corpus.py`

Le collecteur savant interroge Crossref et OpenAlex, fusionne les résultats (`merge(crossref(...), openalex(...))`) et écrit sous verrou. Le collecteur Hugging Face découvre modèles et jeux de données, avec revue de périmètre (`hf-discovery-review.json`, `hf-state.json`).

4.5 Le pipeline hebdomadaire

`weekly_corpus_update.py` orchestre sept étapes, chacune avec un délai maximal de 900 s :

#	Étape	Commande	Critique
1	<code>arxiv</code>	<code>swarm-corpus.py update</code>	Oui
2	<code>crossref_openalex</code>	<code>swarm-scholarly-corpus.py</code>	Oui
3	<code>huggingface</code>	<code>swarm-hf-corpus.py</code>	Non
4	<code>catalog_rebuild</code>	<code>swarm-corpus.py rebuild</code>	Non
5	<code>bibliographic_quality</code>	<code>bibliographic_quality.py update</code>	Non
6	<code>impact_analysis</code>	<code>run_impact_analysis.sh</code>	Non
7	<code>verify_corpus_links</code>	<code>verify_corpus_links.py</code>	Non

Sémantique : une étape critique en échec rend le run global en échec (`success: false`). Une étape non critique en échec produit un run **dégradé** mais valide (`degraded_noncritical`).

Sortie : `last-update-status.json`, avec schéma `weekly-update/v2`, contenant le détail de chaque étape (code de retour, durée, erreur).

Anomalie constatée au 22/09/2026, corrigée le même jour — les étapes 6 et 7 référençaient quatre scripts (`run_impact_analysis.sh`, `analyze_impact.py`, `apply_proposals.py`, `verify_corpus_links.py`) **absents du disque**. Ces étapes avaient fonctionné le 21/09/2026 (traces dans `logs/update-20260921T080122Z.log`) mais les scripts n'avaient jamais été versionnés et ne figuraient dans aucune sauvegarde. **Correction (commit 58f710b)** : les quatre scripts ont été reconstruits à partir de leur comportement observé et de leurs artefacts de sortie (`impact-proposals.json`, `impact-state.json`, `corpus-link-audit.json`), en préservant les interfaces d'origine. 13 tests déterministes ont été ajoutés (`tests/test_impact_and_links.py`). Les deux étapes ont été rejouées avec succès contre le corpus réel, puis synchronisées vers le profil installé (dérive 0). **Niveau de confiance : élevé** (constat direct, recoupé par les journaux ; correction vérifiée par exécution réelle).

4.6 Contrôle qualité bibliographique

`bibliographic_quality.py` maintient un registre (`quality-ledger.json`) et suit trois stades : `DISCOVERED`, `METADATA_VERIFIED`, `PRIMARY_SOURCE_VERIFIED`.

État au 22/09/2026 : 2 139 enregistrements au corpus, 200 au registre (200 `DISCOVERED`, 148 `METADATA_VERIFIED`, 14 `PRIMARY_SOURCE_VERIFIED`), 50 vérifiés lors du dernier run, 52 erreurs signalées.

4.7 Concurrence et verrous

`corpus_lock.py` fournit deux verrous distincts :

- **writer_lock** — verrou d'écriture, tenu uniquement pendant l'écriture

(la découverte réseau s'effectue hors verrou) ;

- **run_lock** — verrou de run, empêchant deux pipelines concurrents.

Le module expose `LockBusy` (verrou déjà tenu) et `LockTimeout` (délai dépassé, avec identification du détenteur : PID et ligne de commande).

Nature des verrous — point à ne pas confondre. Les verrous reposent sur `fcntl.flock` (verrou **noyau**), pas sur la présence du fichier. Conséquence opérationnelle : les fichiers `.run.lock`, `.writer.lock` et `.quality.lock` **restent présents sur le disque après la fin d'un run** — c'est normal. Le verrou est libéré automatiquement par le noyau à la fermeture du descripteur, y compris si le processus est tué. Un fichier `.lock` présent ne signifie donc **pas** qu'un run est en cours ; seul un test `flock` réel permet de le savoir.

Historique : avant le correctif du 21/09/2026, la fenêtre sous verrou couvrait la découverte réseau (mesurée à 655,8 s pour l'étape arXiv) alors que le délai était de 600 s — échec structurellement garanti. Le correctif a réduit la fenêtre de verrou, ajouté le verrou de run et ramené le délai à 300 s.

Incident du 22/09/2026 — un run lancé à 09:00 UTC a été interrompu (SIGKILL externe) à l'étape HuggingFace, après avoir ingéré 105 articles. Le log s'arrête net, sans erreur ni traceback. Le run de reprise (11:38 → 11:46, 455 s) a terminé les 7 étapes avec succès. Le verrou n'était pas en cause : `flock` avait été libéré par le noyau à la mort du processus.

5. Gestion des versions et des dépôts Git

5.1 Inventaire des dépôts

Dépôt	Chemin	Distant	Branche	Com-mits
Profil Swarm-Drone	<code>~/swarmdrone-profile</code>	<code>dagornc/swarmdrone-profile</code> (privé)	master	10
Modèle LikeC4	<code>~/workspace/swarm-drones_likec4</code>	aucun	master	69
Protocole H-Zip	<code>~/workspace/alg-protocole</code>	<code>dagornc/alg-protocole</code> (public)	main	1

5.2 Le modèle de distribution du profil

Le profil suit un modèle **source** → **copie** :

1. La **source** est `~/swarmdrone-profile/` (versionnée).
2. `distribution.yaml` déclare 30 chemins `distribution_owned`.
3. `hermes profile update swarmdrone` reconstruit le payload depuis la source et

écrase ces chemins dans la copie installée.

1. La configuration non secrète et les données propres au profil sont préservées.

Règle opérationnelle : ne jamais éditer la copie installée. Le cycle est :

```
éditer la source → ./scripts/validate-profile.sh → git commit
→ hermes profile update swarmdrone -y
→ python3 scripts/check_profile_drift.py (code 0 = aucune dérive)
```

5.3 Contrôle de dérive

`check_profile_drift.py` compare les 30 chemins possédés par empreinte SHA-256 entre source et copie installée. Codes de sortie : 0 = aucune dérive, 1 = dérive détectée, 2 = erreur d'usage. Option `--json` pour un usage automatisé.

Ce contrôle est exécuté quotidiennement par la tâche cron du §3.3.

5.4 Versionnement du modèle LikeC4

Le dépôt `swarmdrones_likec4` versionne **le savoir** : sources `.c4`, code de vérification, documentation, scripts. Il **exclut le produit régénérable** : sauvegardes `backup_v*/`, exports PNG, `scene.json`, assets binaires.

Justification (documentée dans `docs/versionnement.md`) : un artefact versionné peut mentir sur ce que le code produit réellement — le fichier commité peut dater d'une exécution antérieure au code. Chaque sortie est vérifiable par recalcul, ce qui est un contrôle plus fort qu'un versionnement.

Limite connue : ce dépôt est local, sans distant. Le travail est protégé contre la perte accidentelle de fichiers, pas contre la perte de la machine.

5.5 Le hook `post-commit`

Le dépôt LikeC4 possède un hook `post-commit` qui synchronise automatiquement le modèle vers le répertoire servi après chaque commit.

Comportements garantis :

- ne bloque **jamais** un commit (toujours code 0) ;
- ne synchronise que si le commit touche des `.c4` ou `public/` ;
- appelle **toujours** `sync_likec4.sh --no-restart` : aucun redémarrage

automatique du conteneur ;

- si un `.c4` a changé, affiche « docker restart likec4 requis (autorisation

Christophe) » et s'arrête là.

Désactivation : `LIKEC4_SKIP_SYNC=1 git commit ...` (ponctuelle) ou `chmod -x .git/hooks/post-commit` (durable).

6. Élaboration et stockage des fichiers LikeC4

6.1 Le format LikeC4

LikeC4 est un langage de modélisation d'architecture (C4 model) dont les sources sont des fichiers `.c4`. Le modèle est compilé par l'outil LikeC4 puis servi sous forme d'application web interactive.

Version utilisée : `ghcr.io/likec4/likec4:1.59.2`.

6.2 Les 18 fichiers du modèle

Fichier	Taille	Contenu
<code>architecture.c4</code>	113 870 o	Socle architectural (non réécrit)
<code>views.c4</code>	76 599 o	Définition des vues
<code>algorithms.c4</code>	44 602 o	Catalogue algorithmique
<code>hzip.c4</code>	35 908 o	Protocole H-Zip
<code>e20-audit-completeness.c4</code>	35 517 o	Audit de complétude
<code>metamodel.c4</code>	26 239 o	Métamodèle
<code>functional.c4</code>	17 775 o	Vue fonctionnelle
<code>scenarios.c4</code>	16 084 o	Scénarios
<code>messages.c4</code>	15 370 o	Messages et échanges
<code>worldmodel.c4</code>	14 270 o	Modèle du monde
<code>hardware.c4</code>	13 873 o	Matériel
<code>science.c4</code>	12 638 o	Ancrage scientifique
<code>simulation.c4</code>	10 975 o	Simulation
<code>deployment.c4</code>	9 244 o	Déploiement
<code>facts.c4</code>	7 954 o	Faits
<code>decisions_hw.c4</code>	7 934 o	Décisions matérielles
<code>ux-parcours.c4</code>	6 096 o	Parcours utilisateur
<code>e19-system-context.c4</code>	4 809 o	Contexte système

6.3 Configuration

`likec4.config.json` définit le nom (`swarmdrones`), le titre, et les métadonnées : propriétaire, domaine, périmètre, principe directeur, source et date de génération.

6.4 État du modèle au 22/09/2026

Métrique	Valeur
Éléments	334
Relations	692
Vues	69
Déploiements	2
Types d'éléments	44
Types de relations	31
Descriptions	334/334 (100 %)
Score qualité	99/100

Baseline de gouvernance (`gov_baseline.json`) : 334 éléments, 685 relations, 62 vues, 2 déploiements. L'écart de 7 relations entre la baseline (685) et le comptage courant (692) est **connu et documenté**.

6.5 Contrôles qualité

`tools/qa/integrity_check.py` — 7 contrôles d'intégrité :

ID	Contrôle	Résultat
I-1	Intégrité référentielle	692 relations, 0 arc cassé
I-2	Éléments orphelins	0 orphelin
I-3	Chaîne CAPACITÉ→FONCTION→ALGORITHME	20 liens, 20 avec algorithme
I-4	Contrat de simulation	5 invariants nommés
I-5	Garde-fous World Model	6 règles, 6 testées
I-6	Aucune valeur inventée	0 valeur non sourcée
I-7	Cohérence scientifique	17 constats, 11 verdicts sourcés

`tools/qa/quality_gate.py` — 10 critères pondérés, score 99/100 (cible ≥ 90), `GATE: PASS`. Progression documentée : 31/100 initial $\rightarrow$ 99/100.

6.6 Sauvegardes

22 sauvegardes manuelles (`backup_v1/` ... `backup_v21_final/`, plus `backup_v13_pre_e05integ/`) précèdent l'usage de Git. Elles sont **exclues du versionnement** (doublons) mais conservées sur disque.

7. Élaboration des schémas LikeC4

7.1 Du modèle aux vues

Les schémas sont des **vues** déclarées dans les fichiers `.c4`. Le modèle compte 69 vues, réparties en familles : contexte, conteneurs, déploiement, algorithmes, messages, scénarios, science, monde, matériel, décisions, traçabilité, zones.

7.2 Outillage d'élaboration

Le dépôt contient un ensemble de scripts Python qui transforment, vérifient et enrichissent le modèle :

Script	Rôle
<code>audit_modele.py</code>	Audit global du modèle
<code>audit_comp_alg.py</code>	Audit des liens composant → algorithme
<code>link_algorithms_components.py</code>	Matérialise les liens déclarés mais absents
<code>fix_algorithm_views.py</code>	Corrige les vues algorithmiques
<code>fix_doublons_alg.py</code>	Supprime les doublons
<code>normalise_metadonnees.py</code>	Normalise les métadonnées
<code>refonte_titres_complet.py</code>	Refond les titres
<code>restructure_likec4_nav.py</code>	Restructure la navigation
<code>check_related_components.py</code>	Vérifie les composants liés
<code>verify_served_links.py</code>	Vérifie les liens du modèle servi
<code>add_vue_decisions_hw.py</code>	Ajoute la vue des décisions matérielles

7.3 Leçons apprises (documentées dans `export_likec4.sh`)

1. **Délai Playwright** — le défaut de 15 s est insuffisant pour les grosses vues (ex. `deploiement-30-plateformes` $\approx$ 107 s). Toujours passer `-t 300 --max-attempts 3`.

1. **Fichiers appartenant à root** — le conteneur écrit en root dans `/data` ; nettoyer **depuis le conteneur** (`docker exec likec4 rm -rf ...`).

1. ****include *** est dangereux** — il tire tout ce qui est connecté, y compris les relations de traçabilité. Une vue paysage doit lister ses nœuds ou filtrer par type.

1. **Couleurs** — dans `style { color X }`, `X` est un **nom** de couleur, jamais un code hexadécimal direct.

1. **Rechargement à chaud non fiable** — après modification d'un `.c4`, un `docker restart likec4` est nécessaire si le serveur sert un modèle périmé.

7.4 Export et assets

- `export_likec4.sh` — export fiable du modèle (PNG, JSON)
- `tools/export/export_scene.py` — export de la scène 3D
- `tools/export/verify_render.py` — vérification du rendu
- `tools/assets/build_assets.py` — génération des assets 3D (Blender)
- `tools/assets/render_preview.py` — rendu d'aperçu

Les sorties (`export/scene.json`, `viewer/scene.json`, `assets/*.glb`) sont **exclues du versionnement** et vérifiables par recalcul.

8. Publication vers likec4.breizh.ai

8.1 Architecture de publication

La publication repose sur un **bind mount** :

```
~/workspace/swarmdrones_likec4/  (dépôt Git, développement)
|
|  sync_likec4.sh
▼
/docker/likec4/workspace/        (répertoire servi)
|
|  bind mount
▼
/data  (dans le conteneur likec4)
|
|  likec4 start --public-dir
▼
https://likec4.breizh.ai
```

Conteneur : `likec4`, image `ghcr.io/likec4/likec4:1.59.2`, montage `/docker/likec4/workspace → /data`.
Commande de démarrage :
`likec4 start --listen 0.0.0.0 --port 5173 --no-react-hmr --allowed-host likec4.breizh.ai --public-dir /data/public`.

Mode de service : `likec4 start` est un **serveur de développement Vite**, non un build statique. Le modèle est compilé à la volée à chaque requête ; les sources `.c4` sont lues directement depuis `/data`. Conséquence pratique : une modification de fichier `.c4` est visible sans étape de build, mais le service n'est pas optimisé pour la charge.

État vérifié au 22/09/2026 : site accessible (HTTP 200 en 0,06 s) ; le modèle compilé servi par l'API interne pèse ~5,5 Mo. La mesure antérieure de 11 531 990 octets correspondait à un autre mode de service et n'est plus d'actualité.

8.2 `sync_likec4.sh`

Modes :

- `./sync_likec4.sh` — synchronise et redémarre le conteneur si un `.c4` a changé
- `--dry-run` — montre ce qui serait copié, sans modifier
- `--check` — vérifie l'état de synchronisation (code 1 si divergence)
- `--drift` — vérifie la cohérence `public/` vs servi **et** modèle servi vs

`algorithms.c4`

- `--no-restart` — synchronise sans redémarrer

Garanties de sécurité : ne supprime jamais de fichier dans la destination ; ne copie que les `.c4` et le contenu de `public/` ; sauvegarde horodatée avant chaque copie.

8.3 Le problème du rechargement à chaud

Le serveur de développement LikeC4 (Vite) **ne recharge pas** le modèle de façon fiable quand un `.c4` change (HMR inopérant sur le modèle compilé). Un `docker restart` est nécessaire pour que le site serve le modèle à jour.

Le redémarrage est un acte manuel soumis à l'autorisation explicite de Christophe (service en production). Le hook `post-commit` ne redémarre **jamais** automatiquement.

8.4 Le double cache

Deux niveaux de cache peuvent servir une version périmée :

1. **Index LikeC4** — `likec4 start --public-dir` ne lit les fichiers de

`public/` qu'au **démarrage** du conteneur. Un fichier ajouté après le démarrage n'est pas servi ; un fichier modifié reste servi dans son ancienne version.

1. **Cache CDN** — Cloudflare, `max-age=14400`.

`check_drift()` distingue ces cas en comparant trois vues : le **disque** (ce que le conteneur voit), le **servi** (ce que renvoie l'URL publique), et l'**origine** (requête depuis le conteneur, sans CDN). Un écart disque/servi indique un index périmé (redémarrage requis) ; un écart servi/origine indique un cache CDN transitoire.

8.5 L'angle mort comblé (2026-09-19)

`check_drift` ne comparait que les fichiers de `public/` (PDF). Il ne voyait pas que le **modèle C4 servi** pouvait être périmé alors que `public/` était cohérent. Cas réel constaté : le modèle servi affichait encore « 18,7/20 PUBLIABLE SOUS RESERVE » alors que `algorithms.c4` était à « 19,1/20 PUBLIABLE SANS RESERVE ». Le mode `--drift` a été ajouté pour combler cet angle mort.

8.6 Fichiers publiés

`public/` contient les fichiers servis publiquement :

- `public/consensus_rs/README.md`
- `public/H-Zip_v2.2_Specification_premium.docx`

9. Gestion des spécifications et des algorithmes sur GitHub

9.1 Organisation des dépôts

Le système distingue l'**orchestrateur** des **objets autonomes** :

Dépôt	Visibilité	Rôle
dagornc/SwarmDrones	Public	Dépôt principal de la solution A (orchestrateur)
dagornc/alg-consensus	Public	ALG_CONSENSUS — consensus d'état par CRDT semi-treillis
dagornc/alg-task-allocation	Public	ALG_TASK_ALLOCATION — allocation de tâches
dagornc/alg-protocole	Public	H-Zip v2.2 — protocole de coordination
dagornc/swarmdrone-profile	Privé	Profil Hermes (source de vérité)

9.2 Le principe du submodule

`SwarmDrones` **n'héberge pas** le code des algorithmes ni des protocoles. Il les **agrège** via des Git submodules :

```
[submodule "algorithms/consensus"]
  path = algorithms/consensus
  url = https://github.com/dagornc/alg-consensus.git
[submodule "algorithms/task-allocation"]
  path = algorithms/task-allocation
  url = https://github.com/dagornc/alg-task-allocation.git
[submodule "protocols/hzip"]
  path = protocols/hzip
  url = https://github.com/dagornc/alg-protocole.git
```

Justification (documentée dans le README de `SwarmDrones`) : un algorithme d'essaim est un objet autonome — cycle de vie, tests, parité de référence et versions propres. Le mélanger au code d'orchestration produit un dépôt qui ne peut plus évoluer sans casser autre chose.

Le submodule enregistre un **commit précis**, pas une branche flottante : la solution est donc **re-productible**. Cloner `SwarmDrones` à une date donnée redonne exactement les mêmes versions d'algorithmes.

9.3 Distinction algorithmes / protocoles

- `algorithms/` — les algorithmes de décision (consensus, allocation). Ce sont des **calculs** : ils transforment un état en une décision.
- `protocols/` — les protocoles de communication. Ce sont des ****contrats d'échange**** : ils fixent le format des messages, les fréquences, les règles.

9.4 Structure d'un dépôt d'algorithme

Exemple de `alg-consensus` :

Élément	Rôle
<code>src/</code>	Code Rust
<code>tests/</code>	Tests
<code>reference/</code>	Simulateur Python de référence
<code>Cargo.toml</code> , <code>Cargo.lock</code>	Manifeste et verrouillage Rust
<code>verify_parite_rust.py</code>	Vérification de parité bit-à-bit
<code>README.md</code>	Documentation de référence
<code>LICENSE</code>	Licence

`alg-task-allocation` ajoute `NOTATION_VERSIONS.md` (nomenclature des variantes) et `compare_v1_v2_v3.py` (comparaison des versions).

9.5 Le standard de documentation

Le README de `alg-consensus` fixe le **standard de documentation** des algorithmes publiés : fonctionnement, prérequis, installation, configuration, paramètres d'entrée/sortie, utilisation (CLI et bibliothèque), vérification, limites connues.

9.6 Gestion des versions d'algorithme

`alg-task-allocation` illustre le modèle de versionnement par **sur-ensembles stricts** :

Nom	Drapeaux	Description
v1	<i>(aucun)</i>	CBBA standard : enchère + consensus
v2	<code>evenementiel: true</code>	ED-CBBA : émission si l'état change
v3	<code>evenementiel</code> , <code>resilience</code>	ED-CBBA + heartbeat/quorum + biais appris

Propriété : $v3 \supset v2 \supset v1$. Désactiver les drapeaux de v3 doit reproduire v2 **à l'identique**, et ceux de v2 reproduire v1 **à l'identique**.

Propriétés de non-régression testées :

Propriété	Test	Statut
<code>evenementiel = false</code> $\Rightarrow$ sortie identique à v1	<code>v2_evenementiel_desactive_est_identique_a_v1</code>	✓
<code>resilience = false</code> $\Rightarrow$ aucune détection de partition	<code>v3_resilience_desactivee_ne_detecte_rien</code>	✓
$\beta = 0$ $\Rightarrow$ score identique à v2	<code>beta_zero_est_le_modele_v2</code>	✓
$\alpha = 0$ $\Rightarrow$ score indépendant de l'énergie	<code>alpha_zero_ignore_energie</code>	✓

9.7 Le protocole H-Zip

`alg-protocole` contient la spécification normative du protocole H-Zip v2.2 (`spec/`), avec README et licence. Le protocole est également modélisé dans le modèle LikeC4 (`hzip.c4`, 35 908 o) avec des vues dédiées : `hzipProtocol`, `hzipLayers`, `hzipFrames`, `hzipTests`, `hzipEvidence`.

9.8 Traçabilité entre le modèle et les dépôts

Le modèle LikeC4 matérialise les liens vers les algorithmes et les spécifications :

- 15 éléments de type `algorithm`
- 32 éléments de type `specDoc`
- 3 éléments de type `sourceCode`
- Relations `implements` (32), `implemented-by` (18), `documentedBy` (6)
- Vues dédiées : `algorithmCatalog`, `algorithmCoordination`, `algorithmSafety`, `algorithmSpecTraceability`, `algorithmHosting`

10. Synthèse des risques et points d'attention

10.1 Anomalies constatées

#	Anomalie	Gravité	Confiance
1	~~Étapes 6 et 7 du pipeline corpus référencent 4 scripts absents du disque~~ Corrigé le 22/09/2026 (commit 58f710b)	Moyenne	Élevée
2	Dépôt <code>swarmdrones_likec4</code> sans distant (pas de sauvegarde hors machine)	Moyenne	Élevée
3	Écart de 7 relations entre <code>gov_baseline.json</code> (685) et le comptage (692)	Faible	Élevée
4	52 erreurs signalées dans le registre qualité bibliographique	Faible	Élevée

10.2 Points de vigilance opérationnels

- **Ne jamais éditer la copie installée** du profil : la modification serait

perdue à la prochaine mise à jour. Le contrôle de dérive quotidien détecte cette situation.

- **Perte silencieuse de scripts (corrigée le 22/09/2026)** : `scripts/` étant

distribution-owned, un script présent uniquement dans la copie installée est écrasé à la synchronisation suivante, sans erreur ni trace. C'est la cause racine de la disparition des quatre scripts d'analyse d'impact. Un garde-fou (`tests/test_pipeline_script_references.py`, commit 413a173) vérifie désormais que **tout script référencé par le pipeline existe dans la source versionnée** ; il est exécuté par `validate-profile.sh`. La perte silencieuse est devenue un échec de validation.

- **Le redémarrage du conteneur `likec4` est un acte manuel** soumis à

autorisation explicite : le service est en production.

- **Le double cache** (index LikeC4 + Cloudflare) peut servir une version

périmée de façon transitoire ; `sync_likec4.sh --drift` distingue les cas.

- **Les sorties du pipeline d'export ne sont pas versionnées** : elles sont

vérifiables par recalcul, ce qui est un contrôle plus fort.

10.3 Éléments sensibles

Des identifiants existent dans l'environnement (par exemple dans `~/.hermes/.env`, `~/.hermes/profiles/swarmdrone/config.yaml`, `~/.gitconfig`). **Leurs valeurs ne figurent pas dans ce document** et sont remplacées par `[REDACTED]`. Le script `read-global-deepinfra-secret` lit une clé depuis `~/.hermes/.env` à l'exécution ; il ne contient lui-même aucune valeur.

11. Annexe — Commandes de référence

11.1 Profil

```
# Cycle de modification
cd ~/swarmdrone-profile
./scripts/validate-profile.sh
git commit -am "...
hermes profile update swarmdrone -y
python3 scripts/check_profile_drift.py      # 0 = aucune dérive
```

11.2 Corpus

```
# Audit du corpus
python3 ~/.hermes/profiles/swarmdrone/scripts/swarm-corpus.py \
    --root ~/swarmdrone-research audit

# Mise à jour manuelle
bash ~/.hermes/profiles/swarmdrone/scripts/swarmdrone-corpus-update.sh
```

11.3 Modèle LikeC4

```
cd ~/workspace/swarmdrones_likec4

# Contrôles
python3 tools/qa/integrity_check.py
python3 tools/qa/quality_gate.py
python3 gouvernance_swarmdrones.py

# Synchronisation vers le répertoire servi
./sync_likec4.sh --dry-run
./sync_likec4.sh --check
./sync_likec4.sh --drift
```

11.4 Cron

```
hermes cron list
hermes cron runs <job-id>
hermes cron incidents
hermes cron doctor
```

Fin de la spécification.